Scrum Poker – Projektdokumentation

Modul	Web Technology (Leistungsnachweis «Scrum Poker App»)
Dozent	**TODO Gruppe: Name der Dozentin / des Dozenten eintragen**
Studiengang	**TODO Gruppe: Studiengang und Klasse eintragen**
Gruppenmitglieder	**TODO Gruppe: Vor- und Nachnamen aller Gruppenmitglieder eintragen**
Datum	13. Oktober 2026
Repository	https://github.com/gotazlib/abgaben/tree/main/web-technology/abgabe

Diese Dokumentation erklärt, wie die App funktioniert, wie sie aufgebaut ist und warum sie so gebaut ist. Die Startanleitung in Kurzform steht im README. Verweise der Form `Datei:Zeile` zeigen auf den Code im Projektordner (in der Abgabe der Ordner `code/` bzw. der Inhalt von `scrum-poker-code.zip`). Zur Vorbereitung aufs Fachgespräch liegt dort auch der Leitfaden docs/FACHGESPRAECH.md (auch als PDF und HTML): ein Rundgang durch alle Dateien, Musterantworten, ein Demo-Drehbuch und Übungsaufgaben.

Inhalt

1. Funktionsweise
2. Architektur im Überblick
3. Komponentenstruktur und State
4. Routen und Navigation
5. REST-API
6. Datenfluss: Klick auf eine Karte
7. Lade- und Fehlerzustände
8. Strategie gegen Karteileichen
9. Durchschnitt bei T-Shirt-Grössen
10. Browser-Storage
11. Responsive Design und Box-Modell
12. Tests
13. Lokale Inbetriebnahme
14. Abgabe
15. KI-Einsatz

---

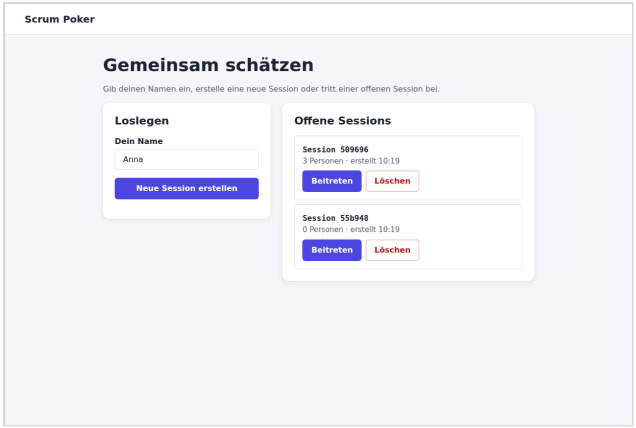
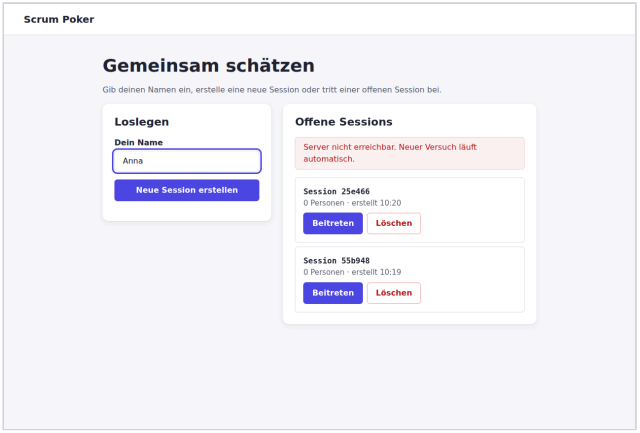
1. Funktionsweise

Scrum Poker (Planning Poker) ist eine Schätzmethode: Alle Teammitglieder legen gleichzeitig und verdeckt eine Karte mit ihrer Schätzung. Erst wenn alle gewählt haben, werden die Karten aufgedeckt. So beeinflusst niemand die anderen.

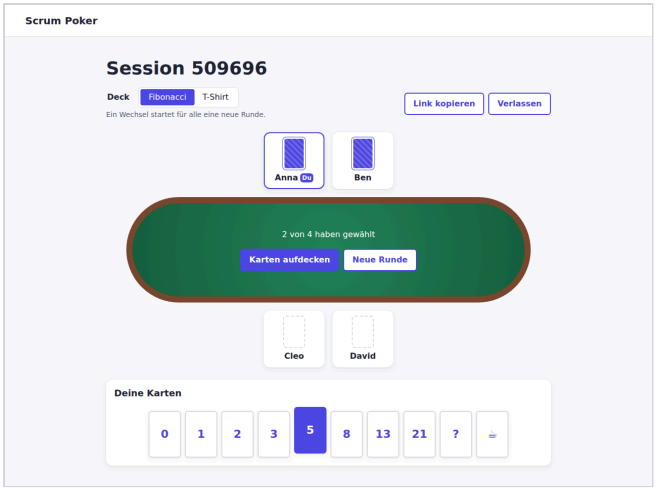
Ablauf in der App

1. **Name eingeben** (Home-Seite `/`). Der Name wird im Browser gespeichert und ist beim nächsten Besuch schon ausgefüllt. Ist der Name leer, besteht er nur aus Leerzeichen oder ist er länger als 30 Zeichen, erscheint eine Meldung und «Neue Session erstellen» und «Beitreten» sind gesperrt.
2. **Session erstellen oder beitreten.** Die Liste «Offene Sessions» lädt sich alle 3 Sekunden neu und zeigt pro Session die Anzahl Personen und die Uhrzeit, zu der sie erstellt wurde. Sessions lassen sich nach einer Rückfrage löschen.
3. **Am Pokertisch** (`/sessions/:sessionId`) sitzen alle Teilnehmenden um einen Tisch. Unten liegen die eigenen Karten. Ein Klick auf eine Karte gibt die Stimme ab, ein zweiter Klick auf dieselbe Karte zieht sie zurück. Die anderen sehen nur eine verdeckte Karte.
4. **Aufdecken:** von Hand mit «Karten aufdecken» oder automatisch, sobald alle gewählt haben. Danach zeigt der Tisch den Durchschnitt, die Anzahl Stimmen und gegebenenfalls «Konsens!».
5. **Neue Runde** löscht alle Stimmen und verdeckt den Tisch wieder.
6. **Deck wechseln** zwischen Fibonacci (0, 1, 2, 3, 5, 8, 13, 21, ?, ☕) und T-Shirt-Grössen (XS, S, M, L, XL, ?). Ein Wechsel startet für alle eine neue Runde.
7. **Link kopieren** kopiert die Adresse der Session. Wer den Link öffnet und noch keinen Namen hat, wird zuerst nach dem Namen gefragt.
8. **Verlassen** führt zurück zur Home-Seite und entfernt die Person sofort aus der Session. Wer den Tab schliesst, wird nach spätestens 20 s entfernt (Kapitel 8). Ein **Reload** wirft niemanden raus: Man sitzt danach als dieselbe Person mit derselben Stimme am Tisch.

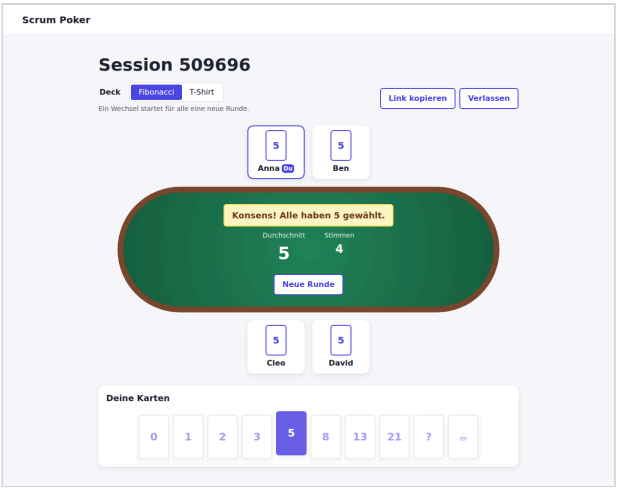
Alle Browserfenster sehen Änderungen der anderen spätestens nach 1.5 Sekunden (Polling).

Home-Seite (Desktop)	Server nicht erreichbar
	

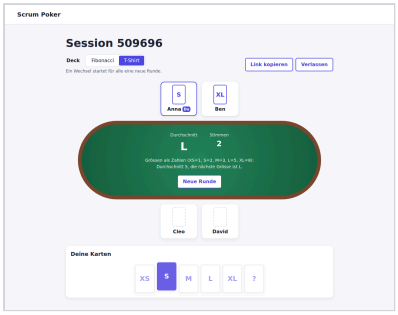
Pokertisch, verdeckt



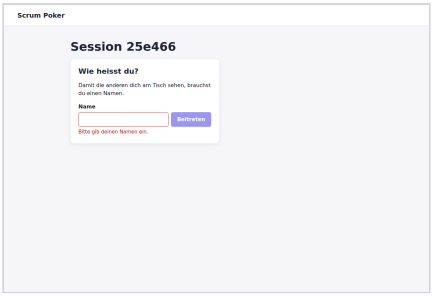
Nach dem Aufdecken mit Konsens



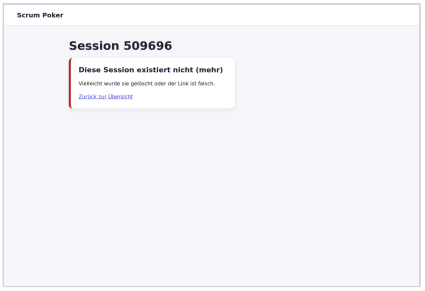
T-Shirt-Deck nach dem Aufdecken



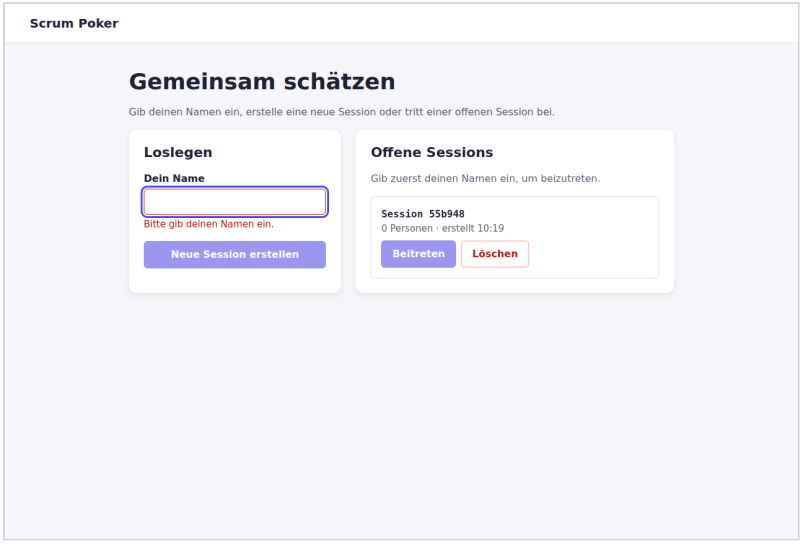
Name fehlt (geteilter Link)



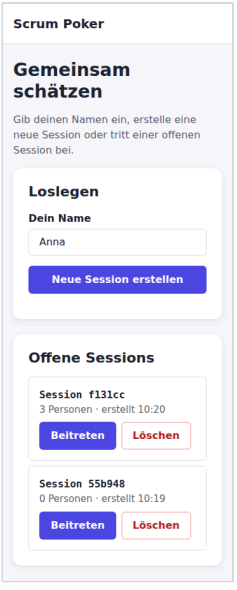
Session gelöscht



Ungültiger Name



Home mobil



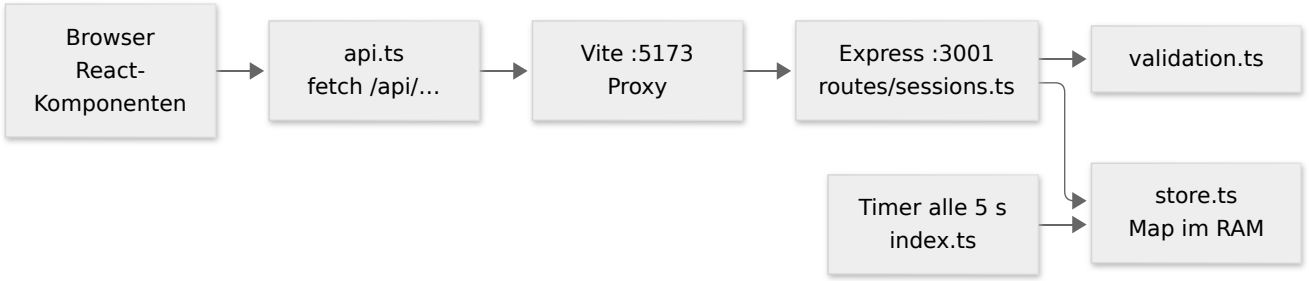
Pokertisch mobil



2. Architektur im Überblick

Das Projekt ist ein **pnpm-Workspace** mit drei Paketen (Dateibaum im README):

Paket	Technik	Aufgabe
`client` (`@scrum-poker/client`)	React 19, TypeScript, Vite, React Router 7	Oberfläche im Browser
`server` (`@scrum-poker/server`)	Node.js, Express 5, TypeScript (`tsx`)	REST-API, Daten im Arbeitsspeicher
`shared` (`@scrum-poker/shared`)	TypeScript, **eine** Datei `shared/src/index.ts`	gemeinsame Types, Decks, `validateName()`, `computeResult()`



Client und Server importieren beide `shared/src/index.ts` über den Paketnamen `@scrum-poker/shared`. `"workspace:*"` in den `package.json` heisst: «nimm den Ordner aus diesem Projekt».

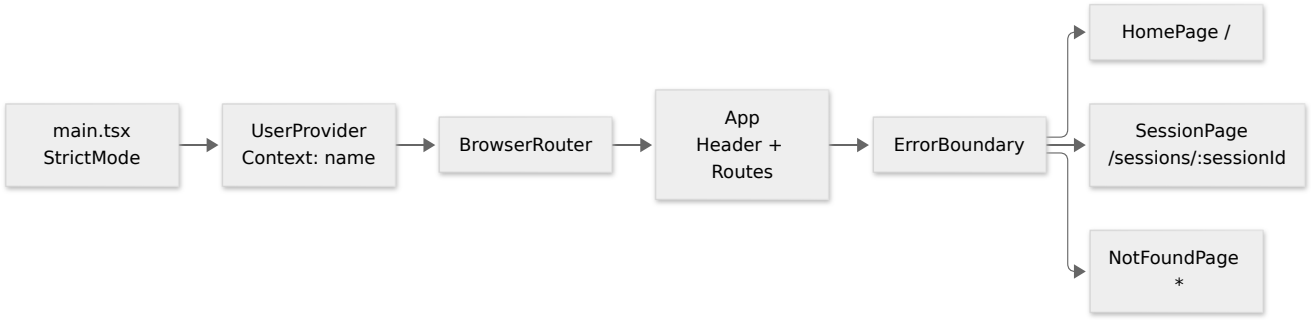
- **Ein Vertrag für beide Seiten.** Alle Types der API (z. B. `SessionState`, `CastVoteRequest`) stehen nur einmal in `shared/src/index.ts`. Ändert sich ein Feld, meldet TypeScript die Fehler im Client **und** im Server. Dasselbe gilt für die Namensregel `validateName()`: Home-Seite, Namensformular am Tisch und Server benutzen dieselbe Funktion.
- **Kein CORS nötig.** Der Client ruft nur relative Adressen wie `/api/sessions` auf. Der Vite-Dev-Server leitet sie an Port 3001 weiter (`client/vite.config.ts:17-21`). Für den Browser gibt es damit nur eine Adresse (`localhost:5173`).
- **Drei Schichten im Server.** Router (HTTP, Statuscodes) → Prüfung der Eingaben → Store (Daten und Regeln, ohne Express). Den Store kann man lesen, ohne Express zu kennen.
- **Daten nur im Arbeitsspeicher.** Eine `Map<string, Session>` (`server/src/store.ts:28`). Nach einem Neustart des Servers sind alle Sessions weg. Für eine Schätzrunde ist das in Ordnung, die Aufgabe verlangt es so.
- **Live-Version.** Dieselbe App läuft unter https://scrumpoker.tasgor.ch. Dort liefert der Express-Server mit `NODE_ENV=production` auch den gebauten Client aus (`server/src/app.ts:19-31`). Für die lokale Entwicklung spielt das keine Rolle.

Bewusst einfach gehalten. Die Gruppe hat den Code nach der ersten Fassung vereinfachen lassen, damit jedes Mitglied jede Stelle erklären kann (Kapitel 15). Darum gibt es zum Beispiel keinen eigenen Polling-Hook, keine eigene Fehlerklasse und keinen `AbortController`: Jede Seite, die pollt, hat ihren eigenen gut lesbaren `useEffect`, und die API-Funktionen werfen normale `Error`-Objekte mit deutscher Meldung.

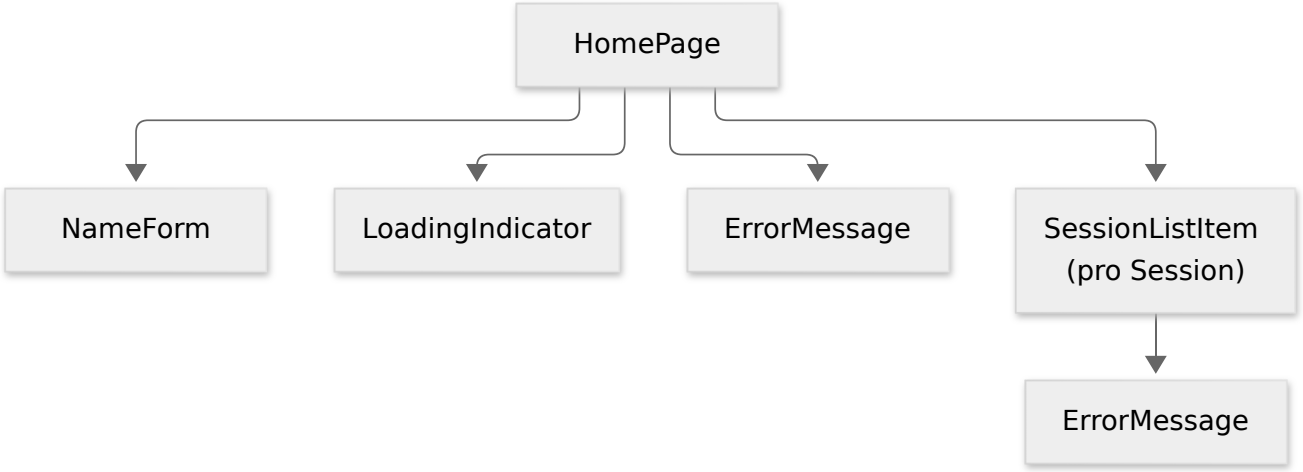
3. Komponentenstruktur und State

3.1 Komponentenbaum

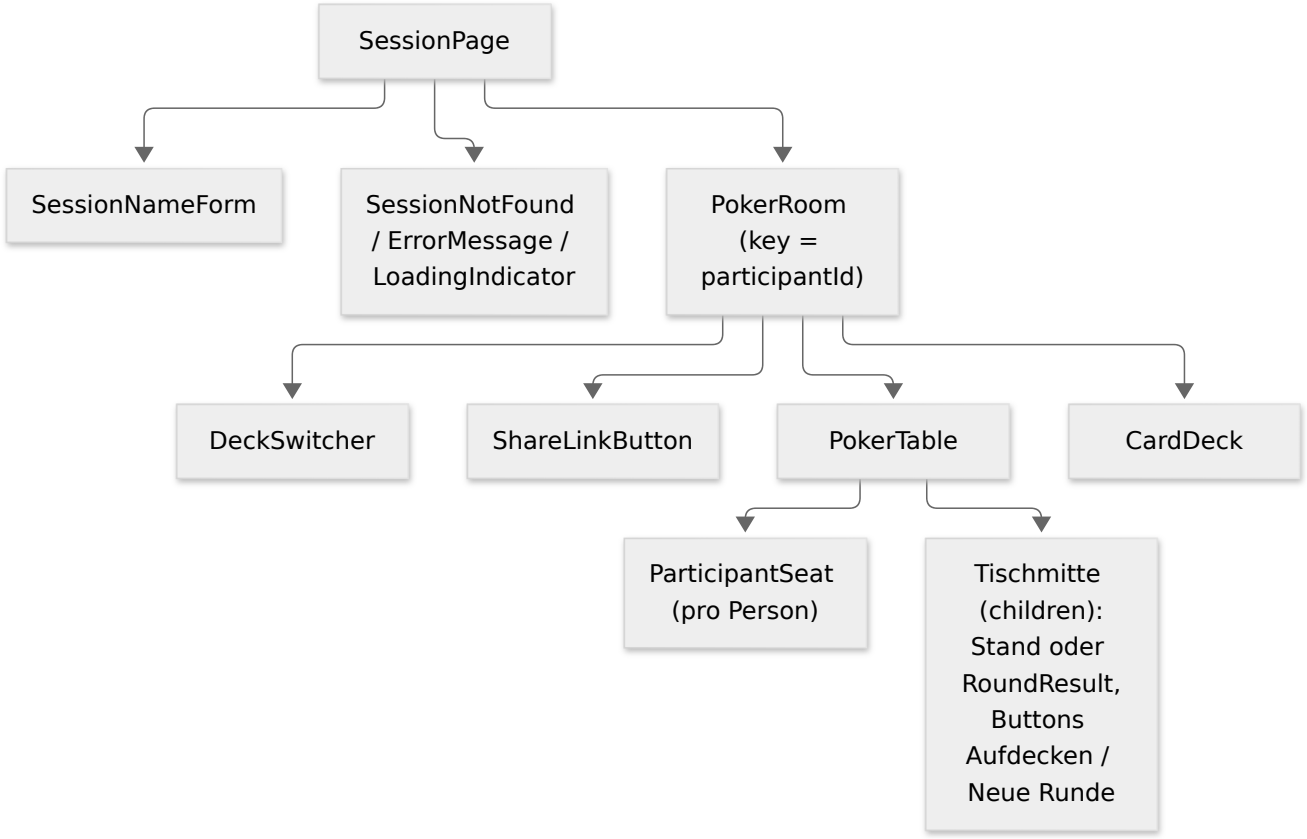
Gerüst und Routen



Home-Seite



Pokertisch



`PokerTable` zeichnet nur die Plätze. Was in der Tischmitte steht, gibt `PokerRoom` als `children` mit (`PokerRoom.tsx:171-185`). So muss `PokerTable` nichts von Aufdecken oder Ergebnis wissen.

3.2 Komponenten, Props und wer sie rendert

Komponente	Gerendert von	Props	Eigener State
`App`	`main.tsx`	–	–
`ErrorBoundary` (einzige Klasse)	`App`	`children`	`hasError`
`HomePage`	Router (`/`)	– (liest den Namen aus dem Context)	`sessions`, `listError`, `isCreating`, `createError`
`NameForm`	`HomePage`	`name`, `error`, `onNameChange`	–
`SessionListItem`	`HomePage`	`session`, `canJoin`, `onDeleted`	`isDeleting`, `deleteError`
`SessionPage`	Router (`/sessions/:sessionId`)	– (`useParams`, Context)	`participantId`, `notFound`, `joinError`, `joinAttempt`
`SessionNameForm`	`SessionPage`	– (schreibt in den Context)	`draft`
`SessionNotFound`	`SessionPage`, `PokerRoom`	–	–
`PokerRoom`	`SessionPage`	`sessionId`, `participantId`, `onRejoin`	`session`, `pollError`, `notFound`, `isBusy`, `actionError`
`DeckSwitcher`	`PokerRoom`	`deck`, `disabled`, `onChange`	–

Komponente	Gerendert von	Props	Eigener State
`ShareLinkButton`	`PokerRoom`	–	`copied`
`PokerTable`	`PokerRoom`	`participants`, `ownId`, `revealed`, `children`	–
`ParticipantSeat`	`PokerTable`	`participant`, `isOwn`, `revealed`	–
`RoundResult`	`PokerRoom` (in der Tischmitte)	`participants`, `deck`	– (wird berechnet)
`CardDeck`	`PokerRoom`	`deck`, `selected`, `disabled`, `onCardClick`	–
`LoadingIndicator`	`HomePage`, `SessionPage`, `PokerRoom`	`label`	–
`ErrorMessage`	`HomePage`, `SessionListItem`, `SessionPage`, `PokerRoom`	`message`, `onRetry?`	–
`NotFoundPage`	Router (`*`)	–	–

Alle Props sind mit einem `interface …Props` typisiert, z. B. `client/src/components/CardDeck.tsx:5-11`. Jede Datei beginnt mit einem kurzen deutschen Kommentar: wozu sie da ist und wer sie benutzt.

3.3 Wo liegt welcher State – und warum?

State	Ort	Begründung
Name	`UserProvider` (`UserContext.tsx:24`), gespeichert in `localStorage`	Wird auf der Home-Seite und am Pokertisch gebraucht. Darum liegt er über dem Router.
Gültigkeit des Namens	nirgends, berechnet mit `validateName(name)` (`HomePage.tsx:24-25`)	Abgeleiteter Wert. Gespeichert könnte er veralten.
Liste der Sessions, `listError`	`HomePage` (`HomePage.tsx:28-29`)	Nur die Home-Seite braucht sie. `SessionListItem` bekommt eine Session über Props.
`isCreating`, `createError`	`HomePage` (`HomePage.tsx:30-31`)	Gehört zum Button «Neue Session erstellen», der direkt in der Seite steht.
`isDeleting`, `deleteError`	`SessionListItem` (`SessionListItem.tsx:21-22`)	Jeder Eintrag hat seinen eigenen Ladezustand beim Löschen.
`participantId`, `notFound`, `joinError`, `joinAttempt`	`SessionPage` (`SessionPage.tsx:25-29`)	Gehört zum Beitreten und Verlassen, das hier im `useEffect` passiert.
Session-Zustand (Teilnehmer, Deck, `revealed`, eigene Stimme)	`PokerRoom` (`PokerRoom.tsx:32`)	**Single Source of Truth ist der Server.** Eigene Karte, Deck und «aufgedeckt» werden daraus gelesen und nicht doppelt gespeichert.
`isBusy`, `actionError`, `pollError`	`PokerRoom` (`PokerRoom.tsx:33-37`)	Ladezustand und Fehler aller Aktionen am Tisch an einer Stelle.
Ergebnis, «x von y haben gewählt»	nirgends, bei jedem Render berechnet	Reine Ableitung aus dem Session-Zustand (`RoundResult.tsx:13-24`, `PokerRoom.tsx:118-120`).
`copied` («Kopiert!»)	`ShareLinkButton`	Betrifft nur diesen Button.

State	Ort	Begründung
`draft` (Namensentwurf)	`SessionNameForm`	Erst beim Absenden kommt der Name in den Context. Sonst würde schon beim ersten Buchstaben beigetreten.

Single Source of Truth. Jede Information liegt an genau einer Stelle. Alles andere wird bei jedem Render daraus berechnet. Ein Beispiel: Die eigene Karte könnte man nach dem Klick in einem eigenen `useState` merken. Dann gäbe es aber zwei Wahrheiten. Klickt eine andere Person auf «Neue Runde», bliebe die Karte bei uns hervorgehoben, obwohl der Server die Stimme schon gelöscht hat. Darum liest `PokerRoom` die eigene Stimme aus der Server-Antwort (`me.vote`, `PokerRoom.tsx:104-118`). Der Server schickt dafür die eigene Stimme schon vor dem Aufdecken mit, die der anderen nicht (`server/src/store.ts:38-40`).

Lifting State Up. Die Komponenten unter `PokerRoom` (`CardDeck`, `DeckSwitcher`, `PokerTable`, `ParticipantSeat`, `RoundResult`) haben keinen eigenen State. Sie bekommen Daten über Props und melden Klicks über Callback-Props nach oben (`onCardClick`, `onChange`). Der State liegt im nächsten gemeinsamen Vorfahren, der ihn braucht: `PokerRoom`. Gleich auf der Home-Seite: `NameForm` ist ein kontrolliertes Eingabefeld (`value={name}`, `onChange` → `onNameChange`), der Wert liegt im Context.

Context für den Namen. Den Namen braucht die Home-Seite (Eingabe) und die Session-Seite (Beitreten, Formular bei geteilten Links). Beide sind Geschwister unter dem Router. Ohne Context müsste `App` den Namen halten und an jede Route weitergeben. Mit Context liest jede Komponente ihn direkt: `const { name, setName } = useContext(UserContext)` (`HomePage.tsx:23`, `SessionPage.tsx:18`). Context, Provider und Speichern stehen zusammen in einer Datei (`client/src/UserContext.tsx`).

Wann sind Props besser? Bei allem anderen in dieser App:

- Daten, die nur eine Ebene nach unten gehen (z. B. eine `session` an `SessionListItem`). Props zeigen, woher die Daten kommen, und die Komponente lässt sich ohne Provider testen.
- Daten, die sich oft ändern, wie der Session-Zustand (alle 1.5 s). Ändert sich ein Context, rendern **alle** Komponenten neu, die ihn lesen. Bei Props rendert nur der betroffene Teil.

Context lohnt sich für wenige, selten geänderte Werte, die viele Stellen auf verschiedenen Ebenen brauchen (Name, Sprache, Farbschema, eingeloggte Person).

`key={participantId}` auf `PokerRoom` (`SessionPage.tsx:112-113`): Nach «Wieder beitreten» gibt es eine neue `participantId`. Der neue `key` lässt React das alte `PokerRoom` wegwerfen und ein frisches mit leerem State anlegen. So wird nie kurz der Zustand des alten Teilnehmers angezeigt.

3.4 Beitreten, Verlassen und React StrictMode

Beitreten und Verlassen passieren an genau einer Stelle: im Effekt von `SessionPage` (`SessionPage.tsx:34-93`).

- **Beitreten** beim Öffnen der Seite mit `joinSession()`. Die Dependencies `[sessionId, trimmedName, hasName, joinAttempt]` sorgen dafür, dass bei einer anderen Session oder einem anderen Namen zuerst verlassen und dann neu beigetreten wird. «Erneut versuchen» und «Wieder beitreten» zählen nur `joinAttempt` hoch (`SessionPage.tsx:95-97`).
- **Verlassen** im Cleanup (`SessionPage.tsx:81-92`). Der Button «Verlassen» navigiert nur auf `/`. Dadurch verschwindet die Seite, und das Cleanup läuft. Es gibt also genau einen Leave-Request, egal wie man den Tisch verlässt (Button, Zurück-Button, Klick auf «Scrum Poker» im Header).

- **Reload:** Bei einem Reload läuft kein Cleanup, man sitzt auf dem Server also noch am Tisch. Nach dem Beitreten merkt sich die Seite die `participantId` in `sessionStorage` (pro Tab, `SessionPage.tsx:69`). Die neu geladene Seite findet sie dort und schickt sie beim Beitreten mit. Kennt der Server die ID noch, antwortet er mit derselben ID (Status 200) statt einen zweiten Teilnehmer anzulegen (`server/src/routes/sessions.ts:96-106`). Platz und Stimme bleiben, und die Runde wird nicht aufgedeckt.
- **Tab schliessen:** Es wird kein Request gesendet. Der Heartbeat-Timeout entfernt die Person (Kapitel 8).
- **Ohne Namen** (geteilter Link in einem neuen Browser) wird nicht beigetreten (`if (!hasName) return`, `SessionPage.tsx:35-37`), sondern `SessionNameForm` gezeigt. Schreibt das Formular den Namen in den Context, ändert sich `hasName`, der Effekt läuft erneut und tritt bei.
- **StrictMode** (`main.tsx:17`) startet in der Entwicklung jeden Effekt beim ersten Anzeigen absichtlich zweimal: Effekt → Cleanup → Effekt. Dadurch gehen zwei Join-Requests los. Das **`ignore`-Flag** (`SessionPage.tsx:49`) ist das Muster aus der React-Dokumentation: Das Cleanup setzt `ignore = true`, und die Antwort des ersten, schon aufgeräumten Effekts wird nicht mehr verwendet. Hat dieser erste Join einen **neuen** Teilnehmer angelegt, verlässt er die Session sofort wieder (`SessionPage.tsx:55-63`). Nach einem Reload haben beide Durchläufe dieselbe gespeicherte ID zurückbekommen. Diese darf nicht verlassen werden, denn der zweite Durchlauf macht mit ihr weiter (`newId !== savedId`). Es bleibt also immer genau ein Teilnehmer. Zwei Tests prüfen das unter `<StrictMode>`.
- **Vom Server entfernt** (z. B. Laptop im Ruhemodus): Fehlt die eigene `participantId` in der Teilnehmerliste, zeigt `PokerRoom` «Du bist nicht mehr am Tisch» mit dem Button «Wieder beitreten» (`PokerRoom.tsx:104-116`).

---

4. Routen und Navigation

Die Routen stehen in `client/src/App.tsx:20-25`:

Pfad	Komponente	Zweck
`/`	`HomePage`	Name, Session erstellen, Liste offener Sessions
`/sessions/:sessionId`	`SessionPage`	Pokertisch einer Session
`*`	`NotFoundPage`	alle anderen Adressen, mit Link zurück

- `App` zeigt den Header und darunter in `<main>` die Seite, die zur Adresse passt. Darum ist der Header auf jeder Seite gleich (`App.tsx:13-27`).
- `BrowserRouter` umschliesst `App` in `main.tsx:19-21`. Er nutzt die History-API des Browsers: echte Adressen ohne `#`, Vor- und Zurück-Button funktionieren.
- Der Header-Link ist ein `<Link to="/">`: Er wechselt die Seite, ohne sie neu zu laden.

Wie kommt die Session-ID in die Komponente? `:sessionId` ist ein URL-Parameter. `SessionPage` liest ihn mit `useParams()` aus (`SessionPage.tsx:17`):

```
const { sessionId = '' } = useParams()
```

`useParams()` liefert `string | undefined`. Der Standardwert `''` macht daraus einen `string`. `SessionPage` gibt die ID als Prop an `PokerRoom` weiter. Weil die ID in der Adresse steht, lässt sich jede Session per Link teilen und neu laden.

Programmatische Navigation mit `useNavigate()`:

Wo	Wann	Ziel
`HomePage.tsx:72`	nach erfolgreichem `POST /api/sessions`	`/sessions/<neue id>`
`SessionListItem.tsx:64`	Klick auf «Beitreten»	`/sessions/<id>`
`PokerRoom.tsx:162`	Klick auf «Verlassen»	`/`

«Beitreten» auf der Home-Seite navigiert nur. Das eigentliche Beitreten (`POST …/participants`) macht `SessionPage` beim Öffnen, denn dort landet man auch über einen geteilten Link.

Interne Links sind immer `<Link>` (Header, `NotFoundPage`, `SessionNotFound`, `ErrorBoundary`), nie `<a href>` oder `window.location`. `window.location.href` wird nur **gelesen**, um den Link zu kopieren (`ShareLinkButton.tsx:20`).

---

5. REST-API

Basis-Adresse: `http://localhost:3001/api` (im Browser über den Proxy `http://localhost:5173/api`). Alle Bodies sind JSON. Jede Fehlerantwort hat die Form `{ "error": "…" }` (`ErrorResponse`). Die Routen stehen in `server/src/routes/sessions.ts`, die Types in `shared/src/index.ts:84-118`.

Methode	Pfad	Body	Antwort	Statuscodes
`GET`	`/api/sessions`	–	`SessionSummary[]`	200
`POST`	`/api/sessions`	–	`{ id }`	201
`DELETE`	`/api/sessions/:sessionId`	–	–	204, 404
`GET`	`/api/sessions/:sessionId?participantId=…`	–	`SessionState`	200, 404
`POST`	`/api/sessions/:sessionId/participants`	`{ name, participantId? }`	`{ participantId }`	201 (neu), 200 (fortgesetzt), 400, 404
`DELETE`	`/api/sessions/:sessionId/participants/:participantId`	–	–	204, 404
`PUT`	`/api/sessions/:sessionId/participants/:participantId/vote`	`{ value: CardValue | null }`	`SessionState`	200, 400, 404, 409
`PUT`	`/api/sessions/:sessionId/deck`	`{ deck: 'fibonacci' | 'tshirt' }`	`SessionState`	200, 400, 404
`POST`	`/api/sessions/:sessionId/reveal`	–	`SessionState`	200, 404
`POST`	`/api/sessions/:sessionId/reset`	–	`SessionState`	200, 404

Für alle Routen gilt zusätzlich: ungültiges JSON → 400, unbekannte Route → 404, unerwarteter Fehler → 500, immer als JSON und nie mit Stacktrace (`server/src/app.ts:33-48`).

Jede Route ist gleich aufgebaut und liest sich von oben nach unten: Session holen → gibt es sie nicht, 404 → Body prüfen (400) → Store ändern → mit Statuscode und JSON antworten. Beispiel: `PUT …/vote` in `sessions.ts:126-152`.

Begründung der HTTP-Methoden

Methode	Verwendet für	Warum
`GET`	Liste, Session-Zustand	Liest nur und darf beliebig oft wiederholt werden. Einzige Nebenwirkung: Mit `participantId` merkt sich der Server «diese Person ist noch da» (`lastSeen`, Heartbeat). Das ist Verwaltung, keine fachliche Änderung.
`POST`	Session erstellen, beitreten	Erstellt etwas Neues, dessen ID der **Server** vergibt → `201 Created`. Zweimal senden erzeugt zwei Einträge. Ausnahme: Beim Beitreten mit einer bekannten `participantId` legt der Server nichts an und antwortet `200 OK`.
`POST`	`reveal`, `reset`	Das sind Aktionen («aufdecken», «neue Runde»), kein Wert, den der Client vollständig setzt.
`PUT`	`vote`, `deck`	Der Client setzt einen Wert vollständig neu. **Idempotent:** Derselbe Request zweimal ergibt denselben Zustand. `value: null` zieht die Stimme zurück.
`DELETE`	Session, Teilnehmer	Entfernt etwas → `204 No Content`. Teilnehmer entfernen ist idempotent: Auch ein schon entfernter Teilnehmer ergibt 204, denn das Ziel («nicht mehr dabei») ist erreicht.

Statuscodes

Code	Bedeutung in dieser API
200	OK, die Antwort enthält Daten (auch: Beitritt nach einem Reload fortgesetzt)
201	Session oder Teilnehmer neu erstellt
204	Gelöscht, kein Body
400	Body ungültig: kein JSON, Name leer oder über 30 Zeichen, Karte nicht im aktiven Deck, unbekanntes Deck
404	Session unbekannt, Teilnehmer unbekannt (beim Abstimmen) oder Route unbekannt
409	Abstimmen, nachdem die Karten aufgedeckt sind (passt nicht zum aktuellen Zustand)
500	Unerwarteter Serverfehler

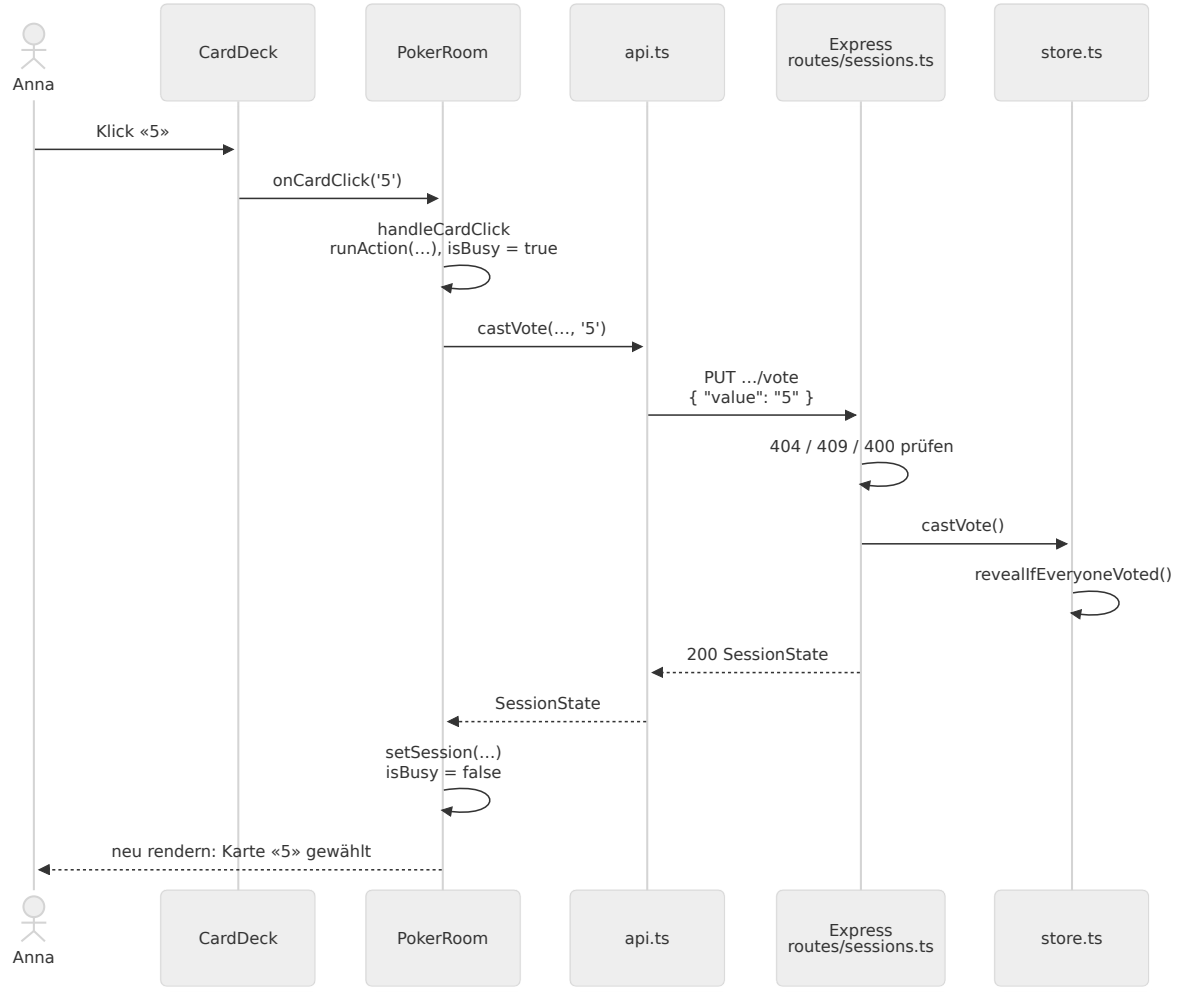
Fachliche Regeln im Server

- **Stimmen bleiben geheim:** Vor dem Aufdecken schickt der Server pro Person nur `hasVoted`, `vote` ist `null`. Ausnahme ist die eigene Stimme der fragenden Person (`store.ts:30-54`). Die Karte der anderen verlässt den Server also nie zu früh, auch nicht in den Entwicklertools des Browsers.
- **Automatisch aufdecken:** Haben alle (mindestens eine Person) gewählt, deckt der Server auf. Das gilt auch, wenn die letzte Person ohne Stimme geht (`store.ts:117-121`, `store.ts:146-157`).
- **Deck-Wechsel** startet eine neue Runde, weil alte Karten im neuen Deck ungültig wären. Dasselbe Deck nochmals wählen ändert nichts (`store.ts:170-177`).
- **Unbekannte `participantId` beim GET** ist kein Fehler (200). Der Client sieht an der Teilnehmerliste selbst, dass er nicht mehr dabei ist, und bietet «Wieder beitreten» an.
- **Eingaben werden immer geprüft**, auch wenn der Client schon prüft. Ein Request kann auch von `curl` kommen. Der Body ist darum `unknown` und wird Schritt für Schritt mit `typeof` geprüft (`server/src/validation.ts`). Erst danach behandelt die Route ihn als typisierten Body (`as JoinSessionRequest`, `sessions.ts:93-94`).
- **Kurze Session-IDs:** 6 Zeichen, damit der Link kurz bleibt. Ist eine ID schon vergeben, wird neu gewürfelt (`store.ts:68-74`).

---

6. Datenfluss: Klick auf eine Karte

Beispiel: Anna klickt im Fibonacci-Deck auf «5». Ben sitzt am selben Tisch und hat noch nicht gewählt.



Schritt für Schritt am Code:

1. **Klick.** Jede Karte ist ein echter `<button>` mit `onClick={() => onCardClick(value)}` (`CardDeck.tsx:23-32`). `CardDeck` kennt nur die Karte, die Entscheidung trifft `PokerRoom` (Lifting State Up).
2. **Entscheidung.** `handleCardClick` vergleicht mit der eigenen Stimme aus dem Server-Zustand: Ist «5» schon gewählt, wird `null` gesendet (zurückziehen), sonst `'5'` (`PokerRoom.tsx:132-139`).
3. **Ladezustand.** `runAction` setzt `isBusy` auf `true` und löscht einen alten Fehler (`PokerRoom.tsx:81-91`). Solange `isBusy` gilt, sind alle Karten und Buttons gesperrt (`disabled={session.revealed || isBusy}`, `PokerRoom.tsx:190`) und in der Tischmitte steht «Wird gesendet …». Ein Doppelklick schickt so keinen zweiten Request.
4. **HTTP-Request.** `castVote()` baut den Body vom Typ `CastVoteRequest` und schickt `PUT /api/sessions/:id/participants/:pid/vote` (`api.ts:126-133`). Die Hilfsfunktion `send()` setzt den Header `Content-Type: application/json` und macht aus «keine Antwort» die Meldung «Server nicht erreichbar.» (`api.ts:21-32`).
5. **Proxy.** Vite leitet `/api/…` an `localhost:3001` weiter (`vite.config.ts:19-21`).

6. **Server.** `express.json()` liest den Body (`app.ts:15`). Die Route (`sessions.ts:126-152`) prüft der Reihe nach: Session vorhanden (sonst 404), Teilnehmer vorhanden (sonst 404), noch nicht aufgedeckt (sonst 409), Karte im aktiven Deck (sonst 400, `validation.ts:32-44`).
7. **Store.** `castVote()` speichert die Stimme und ruft `revealIfEveryoneVoted()` auf (`store.ts:141-157`). Hätte Ben schon gewählt, wäre jetzt `revealed = true`.
8. **Antwort.** Die Route schickt `toSessionState(session, participant.id)` (`sessions.ts:151`). Annas eigener Eintrag enthält `vote: "5"`, alle anderen `vote: null` (`store.ts:38-40`).
9. **Neuer State.** `runAction` legt die Antwort mit `setSession(newSession)` ab und setzt danach `isBusy` zurück (`PokerRoom.tsx:85-90`). Die Antwort wird sofort angezeigt, die App wartet nicht auf den nächsten Poll.
10. **Anzeige.** React rendert `PokerRoom` neu. `myVote` ist jetzt `'5'` (`PokerRoom.tsx:118`), `CardDeck` bekommt `selected="5"`, und die Karte erhält `aria-pressed="true"` und die Klasse `playing-card--selected` (`CardDeck.tsx:26-27`). Annas Platz zeigt eine verdeckte Karte (`ParticipantSeat.tsx:29-31`), und in der Tischmitte steht «1 von 2 haben gewählt» (`PokerRoom.tsx:123-127`).
11. **Andere Browser.** Ben sieht die verdeckte Karte bei seiner nächsten Abfrage (spätestens nach 1.5 s), weil sein `GET` jetzt `hasVoted: true` für Anna liefert.

---

7. Lade- und Fehlerzustände

Alle Requests laufen über `client/src/api.ts`. Die Komponenten rufen nie selbst `fetch()` auf. Jede API-Funktion wirft bei einem Fehler ein normales `Error` mit einer deutschen Meldung. «Diese Session gibt es nicht» (404) ist dagegen **kein** Fehler, sondern der Rückgabewert `null`. Das ist leichter zu lesen als eine Prüfung auf einen Statuscode.

Ursache	Ergebnis der API-Funktion	Stelle
Server läuft nicht, kein Netz (`fetch` wirft)	Fehler «Server nicht erreichbar.»	`api.ts:27-31`
Vite-Proxy erreicht Express nicht (502/503/504)	Fehler «Server nicht erreichbar.»	`api.ts:36-39`
Antwort mit `{ error }` (400, 404 bei Aktionen, 409, 500)	Fehler mit dem Text vom Server	`api.ts:40-45`
Fehlerantwort ohne JSON	Fehler «Anfrage fehlgeschlagen (HTTP …).»	`api.ts:49`
404 bei `getSession` oder `joinSession`	`null` («gibt es nicht»)	`api.ts:88-90`, `api.ts:107-109`
404 bei `deleteSession`	kein Fehler (schon gelöscht, Ziel erreicht)	`api.ts:74-77`

Im `catch` ist der Fehler vom Typ `unknown`. `getErrorMessage(error)` holt daraus den Text für die Anzeige (`api.ts:165-170`).

So behandelt jede Stelle der App die Anfragen:

Situation	Was man sieht	Code
Liste lädt das erste Mal	`LoadingIndicator` «Sessions werden geladen…»	`HomePage.tsx:127`
Server beim Laden der Liste nicht erreichbar	`ErrorMessage` «… Neuer Versuch läuft automatisch.» Die letzte Liste bleibt sichtbar, das Polling versucht es weiter.	`HomePage.tsx:48-53`, `HomePage.tsx:125`

Situation	Was man sieht	Code
Session erstellen läuft	Button gesperrt mit Text «Wird erstellt…»	`HomePage.tsx:111-118`
Session erstellen fehlgeschlagen	Meldung unter dem Button, Button wieder aktiv	`HomePage.tsx:73-76`, `HomePage.tsx:119`
Löschen läuft	Button gesperrt mit Text «Wird gelöscht…»	`SessionListItem.tsx:75`
Löschen fehlgeschlagen	Meldung im Listeneintrag	`SessionListItem.tsx:42-45`, `SessionListItem.tsx:78`
Beitreten läuft	`LoadingIndicator` «Trete der Session bei …»	`SessionPage.tsx:109-111`
Unbekannte Session beim Beitreten	`SessionNotFound` mit Link zur Übersicht	`SessionPage.tsx:64-67`, `SessionPage.tsx:103-105`
Anderer Fehler beim Beitreten	`ErrorMessage` mit «Erneut versuchen» (zählt `joinAttempt` hoch)	`SessionPage.tsx:106-108`
Session wird gelöscht, während man am Tisch sitzt	dieselbe Meldung `SessionNotFound`. Das Polling hört auf.	`PokerRoom.tsx:49-55`, `PokerRoom.tsx:93-95`
Erste Abfrage am Tisch läuft oder schlägt fehl	`LoadingIndicator` «Tisch wird geladen …» bzw. `ErrorMessage`	`PokerRoom.tsx:96-102`
Server antwortet beim Polling nicht	`ErrorMessage` «… Neuer Versuch läuft …». Der letzte Stand des Tisches bleibt sichtbar.	`PokerRoom.tsx:58-63`, `PokerRoom.tsx:168`
Vom Server entfernt (Timeout)	«Du bist nicht mehr am Tisch» + «Wieder beitreten»	`PokerRoom.tsx:104-116`
Aktion läuft (Karte, Deck, Aufdecken, Neue Runde)	«Wird gesendet …» in der Tischmitte, Karten und Buttons gesperrt und ausgegraut, `aria-busy` am Tisch	`PokerRoom.tsx:155`, `PokerRoom.tsx:174`, `PokerRoom.tsx:177-190`
Aktion fehlgeschlagen (z. B. 409 nach dem Aufdecken)	`ErrorMessage`, der Tisch bleibt bedienbar	`PokerRoom.tsx:87-89`, `PokerRoom.tsx:169`
Verlassen	Die Seite wechselt sofort auf `/`, der Leave-Request läuft im Hintergrund. Schlägt er fehl, entfernt der Heartbeat-Timeout die Person, darum wird der Fehler bewusst ignoriert.	`SessionPage.tsx:83-88`
Ungültiger Name (leer, nur Leerzeichen, zu lang)	Meldung unter dem Feld, Buttons gesperrt (`aria-invalid`), auf der Home-Seite und im Namensformular beim geteilten Link	`NameForm.tsx:35-42`, `SessionNameForm.tsx:34-45`, `validateName`
Ungültige Eingabe, die den Client umgeht (`curl`)	Server antwortet 400 mit `{ error }`	`validation.ts`
Unbekannte Adresse im Client	`NotFoundPage`	`App.tsx:24`
Fehler beim Rendern einer Komponente	`ErrorBoundary`: «Etwas ist schiefgelaufen» statt weisser Seite	`ErrorBoundary.tsx:19-38`

Keine unbehandelten Promises. Jede `async`-Funktion fängt ihre Fehler mit `try`/`catch` selbst ab (`loadSessions`, `handleCreate`, `handleDelete`, `join`, `loadSession`, `runAction`, `handleClick` in `ShareLinkButton`). Der Leave-Request im Cleanup hat ein `.catch(() => {})`, weil sein Fehler bewusst egal ist. Darum kann ein Event-Handler eine `async`-Funktion einfach aufrufen, z. B. `onClick={handleCreate}`, ohne dass je eine «Uncaught (in promise)»-Meldung entsteht.

Die ErrorBoundary fängt nur Fehler **beim Rendern** ab, keine Fehler in Event-Handlern oder in `fetch`-Promises. Diese werden darum oben einzeln behandelt. React bietet Error Boundaries nur als Klasse an (`static getDerivedStateFromError`), darum ist sie die einzige Klasse der App.

Späte Antworten. Beide Polling-Effekte (Home und Tisch) haben ein `ignore`-Flag. Das Cleanup setzt es auf `true`. Eine Antwort, die erst danach ankommt, wird nicht mehr in den State geschrieben (`HomePage.tsx:38-63`, `PokerRoom.tsx:41-75`). Einen `AbortController` (Request wirklich abbrechen) verwenden wir bewusst nicht, um die App einfach zu halten. Die einzige Folge: Kommt eine alte Poll-Antwort zufällig erst nach der Antwort einer eigenen Aktion an, zeigt der Tisch für höchstens 1.5 s den alten Stand. Der nächste Poll korrigiert das. Lokal ist das praktisch nie zu sehen.

---

8. Strategie gegen Karteileichen

Problem. Schliesst jemand den Tab, verliert das WLAN oder klappt den Laptop zu, bleibt die Person als «Karteileiche» in der Session. Sie stimmt nie ab, also deckt der Server auch nie automatisch auf.

Die App arbeitet mit **zwei Schichten**: Wer den Tisch in der App verlässt, ist sofort weg. Wer einfach verschwindet (Tab zu, Absturz, Netz weg), wird vom Heartbeat-Timeout entfernt. Ein Reload gehört bewusst zu keiner der beiden: Er soll niemanden aus der Runde werfen.

Schicht 1: sofort verlassen (Button, Navigation in der App)

- «Verlassen» navigiert auf `/`. `SessionPage` verschwindet, und das Cleanup ihres Effekts sendet `DELETE …/participants/:id` (`SessionPage.tsx:81-92`). Dasselbe passiert bei jeder anderen Navigation weg vom Tisch (Zurück-Button, Header-Link).
- Weil nur das Cleanup verlässt, gibt es genau **einen** Leave-Request, egal auf welchem Weg man geht.

Schicht 2: Heartbeat über das Polling, 15 s Timeout, Aufräumen alle 5 s

Beim Schliessen des Tabs, bei einem Absturz, leerem Akku, Netzverlust oder im Ruhemodus läuft kein React-Cleanup, und es wird nichts mehr gesendet. Dafür gibt es den Heartbeat:

1. Der Tisch fragt den Session-Zustand ohnehin alle **1.5 s** ab (`POLL_INTERVAL_MS`, `PokerRoom.tsx:19`) und schickt dabei seine `participantId` mit (`api.ts:86-87`).
2. Der Server wertet jede solche Abfrage als Lebenszeichen und setzt `lastSeen` auf jetzt (`sessions.ts:69-77`).
3. Ein Timer läuft alle **5 s** (`server/src/index.ts:9`, `CLEANUP_INTERVAL_MS` in `server/src/config.ts:10`) und entfernt alle, deren `lastSeen` älter als **15 s** ist (`PARTICIPANT_TIMEOUT_MS`, `config.ts:14`; `store.ts:128-138`).
4. Das Entfernen läuft über `removeParticipant()`. Darum prüft es auch das automatische Aufdecken: War die Karteileiche die Letzte ohne Stimme, wird jetzt aufgedeckt (`store.ts:117-121`).
5. Merkt ein Client, dass er selbst nicht mehr in der Teilnehmerliste steht (z. B. nach dem Ruhemodus), bietet er «Wieder beitreten» an.

Warum diese Zahlen?

- 15 s sind zehn verpasste Abfragen. Ein kurzer Netzaussetzer oder ein langsamer Request wirft niemanden raus.
- Mit 5 s Prüfintervall ist eine verschwundene Person nach spätestens 15 + 5 = **20 s** weg. Das ist kurz genug, dass das Team nicht lange wartet.
- Beide Werte sind benannte Konstanten in `server/src/config.ts`, nicht im Code verstreut.

Reload: dieselbe Person bleibt am Tisch

Ein Reload ruft ebenfalls kein Cleanup auf, die Person bleibt also auf dem Server am Tisch. Die neu geladene Seite schickt beim Beitreten ihre gespeicherte `participantId` mit (Kapitel 3.4 und 10), und der Server macht mit ihr weiter. Weil das Polling nach dem Reload sofort wieder läuft, greift der Timeout nicht. Platz und Stimme bleiben, und es wird nicht automatisch aufgedeckt.

Verworfene Alternativen

Alternative	Warum verworfen
Nur ein Leave-Request beim Schliessen (ohne Heartbeat)	Unzuverlässig: Bei Absturz, leerem Akku, Netzverlust oder Ruhemodus wird nie etwas gesendet.
Zusätzlich Leave beim `pagehide`-Event (Tab schliessen)	War in einer früheren Version eingebaut. `pagehide` feuert aber auch beim **Reload**, und der Browser kann die beiden Fälle nicht unterscheiden. Ein Reload hätte dann die Stimme gelöscht, und war die Person die Letzte ohne Stimme, hätte der Server für alle aufgedeckt. Ein geschlossener Tab bleibt darum höchstens 20 s am Tisch, von Hand aufdecken geht in dieser Zeit trotzdem.
Eigener Heartbeat-Endpunkt (z. B. `POST …/heartbeat` alle 5 s)	Funktioniert, ist aber ein zusätzlicher Request und mehr Code. Das Polling findet sowieso statt und erfüllt denselben Zweck.
Kürzerer Timeout (z. B. 5 s)	Ein Tab im Hintergrund bremst seine Timer. Ein zu kurzer Timeout würde Personen entfernen, die nur kurz das Fenster gewechselt haben.
WebSockets: Verbindungsabbruch = weg	Erkennt Abbrüche sofort, braucht aber eine zweite Technik (WebSocket-Server, Wiederverbinden). Der Kurs und die Aufgabe setzen auf REST und Polling.
Leere Sessions automatisch löschen	Laden alle kurz die Seite neu, wäre die Session sonst weg. Sessions werden nur von Hand gelöscht.
`participantId` in `localStorage` statt `sessionStorage`	`localStorage` teilen sich alle Tabs. Zwei Tabs derselben Person wären dann ein einziger Teilnehmer, und Verlassen in einem Tab würfe auch den anderen raus.

---

9. Durchschnitt bei T-Shirt-Grössen

T-Shirt-Grössen haben keinen Zahlenwert. Gewählte Variante: **jeder Grösse eine Zahl wie bei Fibonacci geben, Durchschnitt rechnen, dann die Grösse anzeigen, deren Zahl am nächsten liegt.**

Grösse	XS	S	M	L	XL
Wert	1	2	3	5	8

Die Tabelle steht als `TSHIRT_VALUES` in `shared/src/index.ts:33-39`. Beispiel: S (2) und XL (8) ergeben 5 → **L**. S (2) und L (5) ergeben 3.5 → am nächsten liegt M (3, Abstand 0.5) → **M**. Unter

dem Ergebnis steht zur Nachvollziehbarkeit «Grössen als Zahlen (XS=1, S=2, M=3, L=5, XL=8): Durchschnitt 5, die nächste Grösse ist L» (`RoundResult.tsx:44-49`).

Begründung

- Die Werte folgen der Fibonacci-Folge wie das Zahlen-Deck. Die Abstände wachsen mit der Grösse, so wie die Unsicherheit bei grossen Aufgaben. Mit 1–5 in gleichen Abständen hätte ein XL zu wenig Gewicht.
- **Gleichstand** (z. B. 4 genau zwischen M=3 und L=5) → die **grössere** Grösse. Beim Schätzen ist Aufrunden die vorsichtigere Wahl. Im Code ist das das `<=` in `shared/src/index.ts:155`: Bei gleichem Abstand gewinnt die später geprüfte, also grössere Grösse.
- `?` und ☕ haben keinen Zahlenwert und zählen nicht zum Durchschnitt, aber zur Anzahl Stimmen. Gibt es gar keinen Zahlenwert, ist der Durchschnitt `null` und es wird «–» angezeigt.
- Der Durchschnitt wird auf eine Nachkommastelle gerundet (4.333 → 4.3, `index.ts:189`).
- **Konsens** heisst: mindestens zwei Stimmen, und alle sind gleich (`index.ts:165-171`).

Verworfene Varianten: den Median oder die häufigste Grösse anzeigen (man sieht nicht mehr, wie weit die Schätzungen auseinanderliegen) oder gar keinen Durchschnitt (die Aufgabe verlangt einen).

Die Berechnung ist die Funktion `computeResult()` (`shared/src/index.ts:163-195`). Sie hängt weder von React noch von Express ab: gleiche Stimmen ergeben immer dasselbe Ergebnis. Darum lässt sie sich ohne Browser testen (`shared/src/index.test.ts`).

---

10. Browser-Storage

	Name	**Teilnehmer-ID**
Speicher	`localStorage` (bleibt nach dem Schliessen des Browsers erhalten)	`sessionStorage` (pro Tab, übersteht einen Reload, nicht das Schliessen des Tabs)
Key	`scrum-poker:name` (`UserContext.tsx:7`)	`scrum-poker:participant:<sessionId>` (`SessionPage.tsx:43`)
Wert	der eingegebene Name	die `participantId` aus der Antwort auf das Beitreten
Zweck	Name ist beim nächsten Besuch schon ausgefüllt, ein geteilter Link braucht keine neue Eingabe	nach einem Reload als dieselbe Person weitermachen (Kapitel 8)

Name (`localStorage`)

- **Lesen:** einmal beim ersten Render des `UserProvider`: `useState(() => localStorage.getItem(NAME_STORAGE_KEY) ?? '')` (`UserContext.tsx:24`). Weil eine Funktion übergeben wird, liest React den Speicher nur beim ersten Render, nicht bei jedem.
- **Schreiben:** `setName()` setzt den State und speichert sofort (`UserContext.tsx:27-30`). Jeder Tastendruck im Namensfeld ist damit gespeichert, auch nach der Rückkehr vom Tisch.
- Der Präfix `scrum-poker:` verhindert Kollisionen mit anderen Apps auf derselben Adresse.
- Getestet in `UserContext.test.tsx` und `HomePage.test.tsx`.

Teilnehmer-ID (`sessionStorage`)

- **Lesen** zu Beginn des Effekts (`SessionPage.tsx:43-44`) und beim Beitreten mitschicken (`api.ts:105`). Ob die ID noch gilt, entscheidet der Server.

- **Schreiben** nach dem Beitreten (`SessionPage.tsx:69`), **Löschen** im Cleanup beim Verlassen (`SessionPage.tsx:87`).
- Getestet in `SessionPage.test.tsx` (Reload mit und ohne gültige ID, auch unter StrictMode) und im E2E-Test (Reload mitten in der Runde).

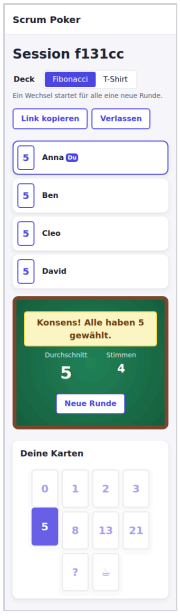
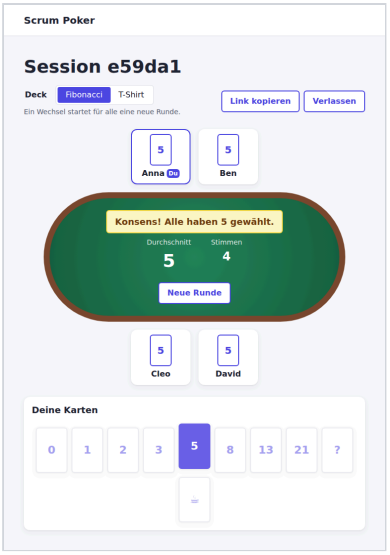
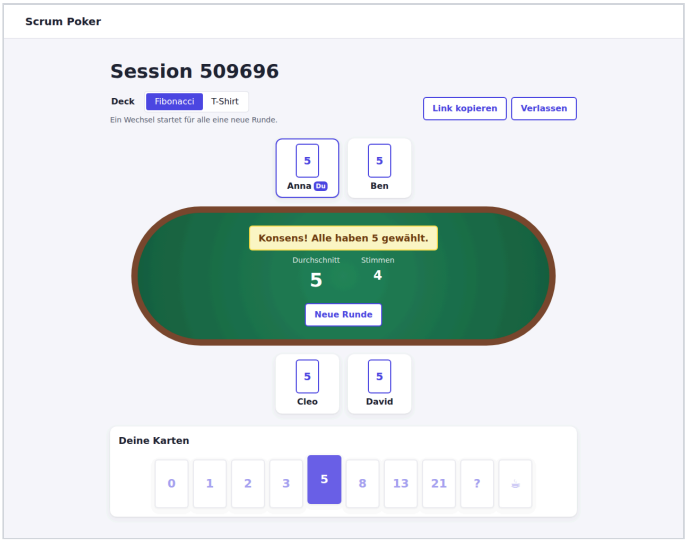
---

11. Responsive Design und Box-Modell

Mobile First. Die Grundregeln gelten für schmale Bildschirme, `@media (min-width: 768px)` ergänzt die Änderungen ab Tablet-Breite. Es gibt einen Breakpoint: **768 px**. Der Client hat drei CSS-Dateien: `styles/global.css` (für alles), `pages/HomePage.css` und `pages/SessionPage.css`.

Bereich	Mobil (< 768 px)	Ab 768 px	Technik
Home-Seite	Karten untereinander	zwei Spalten 2 : 3	**CSS Grid**: `grid-template-columns: minmax(0, 2fr) minmax(0, 3fr)` (`HomePage.css:39-45`)
Pokertisch	alle Plätze als Liste, Tisch darunter (`order: 1`)	Plätze oben und unten, ovaler Tisch dazwischen	**Flexbox** `flex-direction: column` → `row` + `flex-wrap` (`SessionPage.css:144-182`, `SessionPage.css:358-399`)
Teilnehmerplatz	Karte links, Name rechts	Karte über dem Namen	Flexbox (`SessionPage.css:187-197`, `SessionPage.css:370-374`)
Handkarten	brechen in mehrere Zeilen um	grössere Karten	Flexbox mit `flex-wrap: wrap` (`SessionPage.css:313-319`)
Inhalt	Rand `1.5rem 1rem`	Rand `2.5rem`, max. 960 px breit, zentriert	`global.css:109-127`

- Grid für das Seitenlayout mit Spalten, Flexbox für Reihen von Elementen (Buttons, Karten, Plätze), die umbrechen dürfen.
- `minmax(0, …)` verhindert, dass ein langer Inhalt (z. B. ein langer Name) eine Grid-Spalte breiter macht als vorgesehen.
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">` (`client/index.html:6`) sorgt dafür, dass Handys die echte Breite verwenden und die Media Queries greifen.

Mobil (360 px)	Tablet (768 px)	Desktop (1280 px)
		

Box-Modell. Jedes Element besteht aus Inhalt, `padding`, `border` und `margin`. Global gilt `box-sizing: border-box` (`global.css:42-47`): `width` und `height` enthalten Padding und Rahmen. Ein Platz mit `width: 7.5rem` bleibt also 7.5rem breit, egal wie dick sein Rahmen ist. Mit dem Standard `content-box` würden Padding und Rahmen dazugerechnet, und Layouts würden überlaufen. Abstände kommen aus CSS-Variablen (`--space-xs` bis `--space-xl`, `global.css:22-27`), damit sie überall gleich sind.

Reihenfolge der CSS-Dateien. `main.tsx` lädt `global.css` zuerst, danach kommen über `App` die Styles der Seiten (`main.tsx:6-9`). Bei gleich starken Regeln gewinnt die spätere. So kann `HomePage.css` den allgemeinen Button für «Löschen» rot machen (`.button--danger`).

Verdeckte Karten. Ein Platz zeigt eine von drei Karten: leer (gestrichelt), verdeckt (Rückseite mit Muster) oder aufgedeckt (Wert). Das sind drei CSS-Klassen (`SessionPage.css:203-231`). Der Wert der anderen steht vor dem Aufdecken nicht einmal im HTML, weil der Server ihn gar nicht schickt.

Barrierefreiheit (Auswahl): echte `<button>`-Elemente, `aria-pressed` für die gewählte Karte und das aktive Deck, `role="alert"` für Fehler, `role="status"` für Ladeanzeigen, sichtbarer Fokusrahmen (`:focus-visible`), versteckter Status-Text für Screenreader (`.visually-hidden`, z. B. «hat gewählt» am Platz) und `prefers-reduced-motion` schaltet die Drehung der Ladeanzeige aus.

---

12. Tests

Alle Unit-Tests laufen mit **Vitest**. Im Client zusätzlich **React Testing Library** mit **jsdom** (spielt im Test den Browser), im Server **supertest** (HTTP-Requests gegen die Express-App ohne echten Port). Insgesamt 98 Unit-Tests (shared 11, server 42, client 45) und 6 E2E-Tests.

```
pnpm test                              # alle Pakete
pnpm --filter @scrum-poker/server test  # nur ein Paket
pnpm --filter @scrum-poker/client exec vitest   # Client im Watch-Modus
```

Datei	Was getestet wird
`shared/src/index.test.ts`	`validateName` (gültig, leer, nur Leerzeichen, Länge ohne Rand-Leerzeichen); `computeResult`: Durchschnitt Fibonacci, `?`/☕ ignoriert, T-Shirt-Abbildung und Rückrundung, Gleichstand → grösser, kein Zahlenwert → `null`, Konsens erst ab zwei gleichen Stimmen
`server/src/app.test.ts`	alle Routen: erstellen/listen/löschen, 404 auf allen Routen, ungültiges JSON → 400, Namensprüfung, Verlassen idempotent, Fortsetzen mit bekannter `participantId` (200) und neu mit unbekannter (201), Stimmen geheim bis zum Aufdecken, eigene Stimme sichtbar, automatisches Aufdecken (auch beim Verlassen), Stimme zurückziehen, falsche Karte → 400, 409 nach dem Aufdecken, neue Runde, Deck-Wechsel, Heartbeat-Timeout mit Fake-Timern, leere Sessions bleiben
`server/src/static.test.ts`	Produktivbetrieb: `index.html` und Dateien aus `client/dist`, Reload auf `/sessions/<id>`, API funktioniert weiter, ohne `NODE_ENV=production` kein Client
`client/src/App.test.tsx`	Routing: `/`, Session-ID aus der Adresse, Seite «nicht gefunden», Header auf jeder Seite
`client/src/pages/HomePage.test.tsx`	Name aus `localStorage`, sofort gespeichert, ungültiger Name sperrt die Buttons, Session erstellen und navigieren, Fehler beim Erstellen, Ladeanzeige, Sortierung, leere Liste, Server weg, alte Liste bleibt, Polling alle 3 s und Stopp nach dem Verlassen, Beitreten, Löschen mit Rückfrage
`client/src/pages/SessionPage.test.tsx`	Beitreten beim Öffnen und Verlassen beim Schliessen, Reload als dieselbe Person (auch unter StrictMode), unbekannte gespeicherte ID → neu beitreten, Verlassen löscht die gespeicherte ID, StrictMode ohne Duplikat, ohne Namen kein Beitritt, unbekannte und gelöschte Session, Stimme abgeben und zurückziehen, Fehler einer Aktion, verdeckte Karten, Durchschnitt und Konsens, `?`/☕, Deck-Wechsel, Button «Verlassen», «Wieder beitreten», Server nicht erreichbar, «Kopiert!»
`client/src/api.test.ts`	Adresse und JSON-Body, gespeicherte `participantId` beim Beitreten, 404 → `null`, 404 beim Löschen ist Erfolg, Meldung vom Server, Netzwerkfehler und 502 → «Server nicht erreichbar.»
`client/src/UserContext.test.tsx`	Lesen und Schreiben von `localStorage`
`e2e/scrum-poker.spec.ts`	**End-to-End mit Playwright** im echten Chromium gegen den echten Server: zwei Browser schätzen gemeinsam (erstellen, geteilter Link, verdeckt abstimmen inkl. Prüfung, dass die Server-Antwort die fremde Stimme nicht enthält, automatisches Aufdecken, Konsens, Deck-Wechsel, Reload mitten in der Runde ohne Duplikat und ohne Aufdecken, Verlassen, Zurück-Button, Löschen), Tab schliessen entfernt die Person nach dem Heartbeat-Timeout, Fehlerzustände (ungültiger Name, unbekannte Session und Adresse, Server weg), kein horizontales Scrollen bei 360/768/1280 px. Jeder Test prüft, dass die Browser-Konsole keine Fehler oder Warnungen enthält.

Die E2E-Tests starten Server und Client selbst auf eigenen Ports (3101/5273, siehe `playwright.config.ts`) und laufen darum auch neben `pnpm dev`:

```
pnpm exec playwright install chromium   # einmalig (Linux: zusätzlich --with-deps)
pnpm test:e2e
```

Die Client-Tests ersetzen `fetch` durch einen kleinen Fake-Server (`SessionPage.test.tsx:30-83`). So testen sie das Verhalten der Oberfläche, ohne dass ein echter Server laufen muss.

Weitere Prüfungen: `pnpm typecheck` (TypeScript `strict: true` in allen Paketen und in `e2e/`) und `pnpm lint` (ESLint mit `no-explicit-any` als Fehler, schlägt schon bei einer Warnung fehl).

---

13. Lokale Inbetriebnahme

Voraussetzungen

Werkzeug	Version	Prüfen mit
Node.js	**≥ 22.12** (aktuelle LTS empfohlen)	`node -v`
pnpm	10.x	`pnpm -v`
Browser	aktueller Chrome, Firefox, Edge oder Safari	–

Vite 8 und Vitest 5 laufen nicht mehr auf Node 20. Darum ist Node ≥ 22.12 Pflicht (steht auch in `package.json` unter `engines`).

Schritt für Schritt

1. **Node.js installieren** von https://nodejs.org (LTS) oder mit einem Versionsmanager (`nvm install --lts`). Danach `node -v` prüfen.
2. **pnpm aktivieren.** Node bringt Corepack mit, das pnpm bereitstellt:

   ```
   corepack enable
   pnpm -v
   ```

   Corepack liest `"packageManager": "pnpm@10.34.6"` aus `package.json` und verwendet genau diese Version (beim ersten Aufruf Download bestätigen). Ohne Corepack (z. B. Node 25+, wo es nicht mehr mitgeliefert wird) geht auch `npm install -g pnpm@10`.
3. **Projekt holen** – entweder das Code-ZIP entpacken oder das Repository klonen – und in den Projektordner wechseln:

   ```
   cd scrum-poker           # aus dem ZIP
   # oder: cd abgaben/web-technology/abgabe/code
   ```

4. **Abhängigkeiten installieren** (einmalig):

   ```
   pnpm install
   ```

5. **Server und Client starten:**

   ```
   pnpm dev
   ```

   Im Terminal erscheinen zwei farbige Präfixe: `[server]` meldet `Scrum Poker API läuft auf http://localhost:3001`, `[client]` meldet die Vite-Adresse.
6. **Im Browser öffnen:** http://localhost:5173. Zum Ausprobieren ein zweites Fenster (oder ein privates Fenster mit anderem Namen) öffnen und derselben Session beitreten.

Dienst	Adresse	Port
Client (Vite-Dev-Server)	http://localhost:5173	5173

Dienst	Adresse	Port
API (Express)	http://localhost:3001/api/sessions	3001
API über den Proxy	http://localhost:5173/api/sessions	5173

Weitere Befehle: `pnpm test`, `pnpm lint`, `pnpm typecheck`, `pnpm build` (siehe README). Beenden mit `Ctrl+C` im Terminal.

Troubleshooting

Problem	Ursache und Lösung
`Port 5173 is already in use`	Ein anderer Vite-Server läuft noch (`strictPort: true` verhindert ein stilles Ausweichen auf 5174, denn dann stimmte die Adresse nicht mehr). Alten Prozess beenden: `lsof -i :5173` (macOS/Linux) bzw. `netstat -ano | findstr :5173` (Windows) und den Prozess schliessen. Oder andere Ports wählen, siehe unten.
`EADDRINUSE: address already in use :::3001`	Ein alter Server läuft noch. Wie oben mit Port 3001.
Ports dauerhaft belegt	Ports per Umgebungsvariable ändern: `PORT=3002 API_PORT=3002 CLIENT_PORT=5174 pnpm dev` (macOS/Linux; Windows PowerShell: `$env:PORT=3002; $env:API_PORT=3002; $env:CLIENT_PORT=5174; pnpm dev`). `PORT` gilt für Express, `API_PORT` sagt dem Vite-Proxy, wohin er weiterleiten soll.
Rote Meldung «Server nicht erreichbar.»	Der Express-Server läuft nicht oder ist abgestürzt. Im Terminal nach `[server]`-Fehlern schauen, `pnpm dev` neu starten.
`Unsupported engine` / Syntaxfehler beim Start	Node-Version zu alt. `node -v` muss ≥ 22.12 sein.
`pnpm: command not found`	Schritt 2 (Corepack) ausführen oder Terminal neu öffnen.
`ERR_PNPM_…` bei `pnpm install`	Mit `npm` oder `yarn` installiert? `node_modules` löschen und nur `pnpm install` verwenden.
Alle Sessions weg	Der Server hat neu gestartet (z. B. nach einer Code-Änderung durch `tsx watch`). Die Daten liegen nur im Arbeitsspeicher, das ist so gewollt.
Eine Person bleibt nach dem Schliessen des Tabs oder einem Absturz am Tisch	Sie verschwindet nach höchstens 20 s (Heartbeat-Timeout, Kapitel 8).

---

14. Abgabe

Zwei gleichwertige Wege:

1. **ZIP-Datei:** Im Projektordner

   ```
   ./scripts/make-submission.sh
   ```

   ausführen. Es erzeugt `scrum-poker-code.zip` mit `git archive` aus dem committeten Stand: den Quellcode ohne `node_modules` und `dist`, dazu den Leitfaden fürs Fachgespräch (`docs/FACHGESPRAECH.md` / `.pdf` / `.html`). Diese Dokumentation (PDF und HTML) wird daneben abgegeben. Nach dem Entpacken laufen `pnpm install`, `pnpm test` und `pnpm build` ohne weitere Schritte. Das ZIP selbst ist nicht im Git (`.gitignore`).

2. **Git-Link** auf den öffentlichen Abgabe-Ordner (Dokumentation, Leitfaden, ZIP und Quellcode): https://github.com/gotazlib/abgaben/tree/main/web-technology/abgabe.

PDF und HTML dieser Dokumentation und des Leitfadens werden mit `pnpm docs:pdf` aus den Markdown-Dateien erzeugt (nach jeder Änderung erneut ausführen).

Bearbeitbare Version: `DOKUMENTATION.docx`. Nach dem Bearbeiten in Word Inhaltsverzeichnis aktualisieren und als PDF speichern. Die Word-Datei nutzt die HWZ-Vorlage und wird mit `pnpm docs:docx` aus dieser Markdown-Datei erzeugt. Das überschreibt Änderungen, die in Word gemacht wurden.

---

15. KI-Einsatz

Offenlegung

Die App wurde zum grössten Teil mit KI generiert. Code, Tests, CSS, Screenshots und diese Dokumentation hat ein KI-Agent geschrieben. Die Gruppe hat die Aufgaben formuliert, Anforderungen aus der Aufgabenstellung vorgegeben, Ergebnisse geprüft und die Änderungen freigegeben.

Werkzeug	Modell	Einsatz
Claude Code (CLI von Anthropic), mehrere Agenten mit je einem Pull Request pro Arbeitspaket	Claude Opus	gesamte Umsetzung, siehe unten

Arbeitspaket	Was die KI gemacht hat
Grundgerüst	pnpm-Workspace, Vite-/Express-Gerüst, TypeScript- und ESLint-Konfiguration, gemeinsame Types in `shared`
Server	REST-API, Store, Validierung, Heartbeat-Strategie, Server-Tests
Home-Seite	Komponenten, Context, Polling, Löschen, Tests, Screenshots
Pokertisch	Beitreten/Verlassen mit StrictMode, Karten, Decks, Aufdecken, Durchschnitt, Tests, Screenshots
CSS	Mobile-First-Layout, Tisch, Karten
Abnahme	Prüfung gegen die Aufgabenstellung, Ports per Umgebungsvariable, E2E-Tests mit Playwright, Screenshots für Desktop/Tablet/Mobile
Dokumentation	diese Dokumentation, Leitfaden fürs Fachgespräch `docs/FACHGESPRAECH.md`, PDF-, HTML-, DOCX- und Abgabe-Scripts
Endkontrolle	Prüfung jeder Anforderung mit Checkliste; Reload setzt als dieselbe Person fort (`sessionStorage`), Namensprüfung auch beim geteilten Link sichtbar, Doku nachgeführt
Vereinfachung (auf Wunsch der Gruppe)	Die Gruppe hat zurückgemeldet, dass der erste Code für Anfänger zu schwer zu erklären war. Auf ihren Wunsch hat die KI einen Vereinfachungsplan geschrieben (nur Kursstoff, keine eigenen Hooks, Generics oder cleveren Abstraktionen) und den Code in drei Teilen danach umgebaut: Server und gemeinsame Types, Client-Basis und Home-Seite, Pokertisch. Ein KI-Reviewer hat jeden Teil aus Sicht eines Anfängers und gegen die Anforderungs-Checkliste geprüft. Ergebnis: weniger Dateien (z. B. ein `api.ts` statt Ordner `api/`, `hooks/`, `utils/`, `context/`), jede Datei mit deutschem Kommentar am Anfang, gleiche Funktionalität und alle Tests weiterhin grün. Danach wurden diese Dokumentation, der Leitfaden und die Screenshots an den neuen Code angepasst.

Alle Arbeitsschritte sind in der Git-Historie nachvollziehbar.

Von Hand geschrieben oder angepasst

TODO Gruppe: Hier ehrlich auflisten, was ihr selbst geschrieben, geändert oder entschieden habt (Datei, Art der Änderung). Wenn nichts von Hand geschrieben wurde, das so stehen lassen und stattdessen beschreiben, was ihr geprüft und entschieden habt (z. B. den Wunsch nach einfacherem Code). Nichts erfinden.

Wie die Gruppe sichergestellt hat, dass sie den Code versteht

TODO Gruppe: Beschreiben, wie ihr den Code durchgearbeitet habt (z. B. Leitfaden docs/FACHGESPRAECH.md mit der Aufteilung im Kapitel «Aufteilung für die Gruppe» durchgearbeitet, Übungsaufgaben gelöst, Code gemeinsam besprochen).

Gruppenmitglieder und Aufteilung

TODO Gruppe: Namen aller Gruppenmitglieder und wer welchen Teil des Codes zuerst vorbereitet hat. Im Fachgespräch muss trotzdem jede Person alles erklären können.